

Engineering Cognition: Technical Report 001

First Evaluation of Engineering Cognition using EC-Bench

By: Mudit Sarda
Date: 15 July, 2026

1) Abstract

Engineering Cognition is the hypothesis that AI coding agents should not merely retrieve information from previous sessions but also accumulate engineering understanding to improve their software engineering capabilities over time. While recent work on coding agents has significantly improved code generation, we observed that long-running software engineering tasks remain bottlenecked by the repeated reconstruction of architectural context, implementation rationale, debugging discoveries, and repository-specific engineering knowledge. Existing memory systems largely treat this as a retrieval problem, preserving conversations, summaries, or embeddings. We hypothesised that the underlying problem is instead one of accumulated engineering cognition.

To investigate this hypothesis, we simultaneously developed two research artefacts. The first is the **Engineering Memory System (EMS)**, an experimental platform for extracting, storing, and reusing engineering understanding across development sessions. The second is **EC-Bench**, a benchmark designed to measure whether accumulated engineering cognition changes how AI agents perform software engineering tasks. Rather than assuming the hypothesis to be true, EC-Bench evaluates identical engineering workflows with and without access to accumulated engineering cognition while holding every other experimental variable constant.

The results of the first experiment were both encouraging and unexpectedly corrective. Across investigation, planning, and architectural reasoning tasks, accumulated engineering cognition consistently improved engineering behaviour compared to a stateless coding agent. At the same time, implementation and verification tasks revealed several flaws in the initial EMS architecture. The benchmark demonstrated that retrieving engineering understanding before repository reasoning often degraded performance, leading to a redesigned, demand-driven retrieval strategy. It also exposed previously unrecognised research problems, including engineering belief evolution, task-mode inference, scope discipline, and the distinction between observation and intervention during engineering workflows.

Perhaps the most important outcome of this experiment was not the performance difference itself, but the fact that EC-Bench repeatedly invalidated several of our own assumptions. As a

result, both EMS and EC-Bench evolved throughout the experiment, reinforcing our belief that engineering cognition should be approached as an iterative research problem rather than a solved engineering system.

Rather than presenting a finished system, this report captures the current state of our understanding. Many of the ideas described here will almost certainly evolve as future experiments invalidate or refine the hypotheses presented in this work.

2) Background

2.1) Why Engineering Cognition?

The rapid progress of modern AI coding agents has significantly reduced the cost of code generation. Models are now capable of understanding large repositories, generating substantial code changes, and autonomously executing many software engineering tasks that previously required continuous human involvement. As these capabilities improved, we observed that code generation itself gradually ceased to be the primary bottleneck during day-to-day software development.

Instead, long-running engineering tasks increasingly became limited by the repeated reconstruction of engineering understanding. Every new coding session required the agent to rediscover architectural assumptions, implementation rationale, debugging discoveries, repository conventions, and engineering decisions that had already been established during previous work. While human engineers naturally accumulate this understanding over weeks or months of development, contemporary coding agents largely rebuild it from scratch at the beginning of each session.

Current approaches typically frame this limitation as a memory problem. Conversation persistence, retrieval systems, embeddings, repository indexing, and compressed summaries all attempt to preserve previously generated information so that it can be retrieved later when required. These approaches have proven valuable, but they primarily operate on stored artefacts rather than on accumulated engineering understanding itself.

The Engineering Cognition hypothesis proposes a different framing. Rather than asking how an agent should remember previous interactions, it asks whether engineering understanding itself can become an explicit computational object that accumulates, evolves, and influences future engineering decisions in the same way that accumulated experience influences human engineers.

The theoretical motivation, terminology, and conceptual framework behind Engineering Cognition are presented in the accompanying *Engineering Cognition* paper. The purpose of the

present report is not to re-establish that hypothesis, but to evaluate whether it produces measurable changes in software engineering behaviour when implemented within an experimental system.

2.2) Why this experiment?

The Engineering Cognition hypothesis proposes that accumulated engineering understanding should improve the quality of long-running software engineering performed by AI agents. While the idea appeared intuitively compelling during the development of EMS, intuition alone provides little evidence that such a system produces measurable improvements in engineering behaviour.

More importantly, an implementation that appears successful in isolated demonstrations can easily reinforce incorrect assumptions. Without systematic evaluation, it becomes difficult to distinguish genuine improvements from confirmation bias, prompt engineering, or implementation-specific behaviour. We therefore considered experimental evaluation to be a necessary component of the research process rather than a validation exercise performed after the system was complete.

At the time this work began, we were unable to identify an existing benchmark designed to evaluate accumulated engineering understanding. Most software engineering benchmarks focus on code generation, issue resolution, or functional correctness. While these are valuable measurements, they do not capture the central question motivating this research:

Does accumulated engineering understanding change how an AI agent performs software engineering over time?

Answering this question required an experimental framework capable of comparing engineering behaviour under controlled conditions while varying only a single factor: the availability of accumulated engineering cognition.

This led to the development of **EC-Bench**, a benchmark specifically designed to evaluate long-horizon software engineering tasks performed both with and without Engineering Cognition. Rather than attempting to demonstrate that EMS succeeds, the benchmark was designed to challenge its underlying assumptions. Wherever possible, the experiment sought situations in which accumulated engineering understanding could fail, reveal weaknesses in the system, or invalidate our existing beliefs about how Engineering Cognition should operate.

Consequently, the objective of this experiment was not simply to measure whether EMS performed better than a stateless baseline. The primary objective was to understand *why* differences emerged, which assumptions proved correct, which assumptions failed, and how those observations should influence the future evolution of both EMS and the broader Engineering Cognition research program.

3) Research Philosophy

Engineering Cognition is an attempt to investigate a question that, to the best of our knowledge, has not yet been studied systematically. As a consequence, we do not expect our first hypotheses, implementations, or experimental designs to be entirely correct. We view this as a natural property of exploratory research rather than a shortcoming of the process.

The work presented in this report originates from software engineering rather than from a traditional machine learning research environment. Our background is in building distributed systems, developer infrastructure, and production software. This perspective inevitably influences both the questions we ask and the methods we use to investigate them. At the same time, we recognise that intuition gained from engineering experience is not evidence. Wherever possible, our objective is to replace intuition with observation.

For this reason, every experiment conducted within EC Labs is designed with a single guiding principle: **the experiment exists to challenge the hypothesis, not to defend it**. If an experiment disproves one of our assumptions, we consider that outcome at least as valuable as a successful result. A hypothesis that survives careful attempts at invalidation becomes more credible; one that fails provides an opportunity to refine our understanding of the problem.

Accordingly, we make a conscious effort to document both positive and negative outcomes. Improvements are recorded, but so are incorrect assumptions, architectural mistakes, benchmark limitations, and experimental failures. In our view, these observations are not incidental details to be hidden between iterations; they are part of the research record itself. Every mistake contributes information about the boundaries of the current understanding.

This philosophy also extends to the tools we are building. Neither EMS nor EC-Bench should be viewed as fixed systems. Both are research artefacts that are expected to evolve as experiments reveal weaknesses in their underlying assumptions. We fully expect future iterations to revise, replace, or even abandon ideas presented in this report if empirical evidence suggests better alternatives.

Ultimately, our goal is not to demonstrate that Engineering Cognition is correct. Our goal is to understand the problem more accurately after every experiment than we did before it. If each iteration leaves us with fewer assumptions and better questions, then the research is moving in the right direction.

4) Initial Hypothesis

Before conducting the first EC-Bench experiment, Engineering Cognition existed primarily as a conceptual framework supported by engineering intuition rather than empirical evidence. The hypotheses presented below capture the assumptions that guided the design of EMS, the

formulation of Engineering Cognition, and the construction of EC-Bench. They represent what we believed *before* the experimental results became available.

Where appropriate, these hypotheses intentionally include assumptions that were later shown to be incomplete or incorrect. They are preserved here because understanding how our thinking evolved is an important part of the research record.

4.1) EMS Hypothesis

The earliest version of EMS was motivated by the observation that AI coding agents repeatedly reconstructed engineering context across sessions. At that stage, we believed the primary limitation was the absence of persistent engineering memory.

Our initial assumptions were:

- Engineering productivity would improve if important engineering knowledge could persist across sessions instead of being reconstructed repeatedly.
- Repository-specific engineering knowledge should be accumulated continuously rather than recreated from chat history.
- Engineering knowledge should be organised hierarchically (repository → branch → session) because software engineering itself naturally evolves at multiple levels of abstraction.
- Developers should remain the final authority over what becomes permanent engineering knowledge, implying that accumulated knowledge should be reviewed rather than accepted automatically.
- A persistent engineering knowledge base would enable future coding sessions to begin from accumulated understanding rather than from an empty context.

At this stage, we viewed the problem primarily as one of **persistent memory**.

4.2) Engineering Cognition Hypotheses

As EMS evolved, we gradually concluded that persistence alone was insufficient. The problem appeared less related to remembering previous conversations and more related to accumulating engineering understanding itself.

This shift led to several new hypotheses.

- Engineering understanding is fundamentally different from stored information.
- Architectural intuition, implementation rationale, debugging discoveries, and engineering assumptions together constitute an evolving form of engineering cognition.
- Engineering cognition should influence future engineering decisions rather than simply serving as retrieved reference material.

- Engineering understanding should evolve instead of remaining a static collection of documents or summaries.
- Long-running software engineering agents should eventually accumulate engineering understanding in a manner analogous to experienced human engineers.

These hypotheses motivated the transition from an Engineering Memory System toward a broader research program centred around Engineering Cognition.

4.3) Experimental Hypotheses

Immediately before conducting the first EC-Bench experiment, we believed that introducing Engineering Cognition into the engineering workflow would produce measurable improvements across multiple stages of software development.

Specifically, we expected that:

- Engineering Cognition would improve repository investigation by reducing redundant exploration.
- Engineering Cognition would improve architectural reasoning by allowing previous engineering decisions to influence future planning.
- Engineering Cognition would improve implementation quality through better engineering context.
- Engineering Cognition would reduce unnecessary debugging effort by preserving previously discovered engineering knowledge.
- Engineering Cognition would improve overall engineering quality across the complete software engineering workflow.

Perhaps the strongest assumption underlying EMS at the time concerned **when** engineering cognition should be introduced into the reasoning process.

We believed that:

- Engineering Cognition should be retrieved **before** the agent begins reasoning about the engineering task.

Our intuition was straightforward. If an experienced engineer already possesses relevant understanding before approaching a problem, then providing accumulated engineering cognition at the beginning of the reasoning process should enable the agent to make better engineering decisions throughout the remainder of the workflow.

This assumption directly influenced the initial architecture of EMS and became one of the central ideas evaluated during the first EC-Bench experiment.

5) Experiment Design

5.1) Goal

The primary objective of this experiment was to determine whether accumulated engineering cognition measurably changes the behaviour of an AI software engineer during a realistic long-horizon development task.

Rather than evaluating code generation in isolation, we sought to observe how an engineering agent behaves while carrying out the complete lifecycle of a feature implementation: understanding an unfamiliar repository, making architectural decisions, implementing new functionality, debugging regressions, validating the implementation, and finally documenting the work. The experiment was designed to answer a simple but fundamental question: **does accumulated engineering understanding improve software engineering, or does it merely change the information available to the model?**

Equally important, the experiment was intended to evaluate EMS itself. We did not view EMS as a finished system whose effectiveness needed to be demonstrated. Instead, it served as an experimental environment through which different hypotheses about engineering cognition could be tested. Every observation—positive or negative—was therefore considered evidence for improving both the underlying cognition model and the infrastructure responsible for retrieving and presenting it.

5.2) Independent Variable

The experiment compared two nearly identical software engineering sessions that differed in only a single aspect: the availability of accumulated engineering cognition.

The **baseline condition** consisted of a repository containing only the source code. The agent had access to the repository exactly as any stateless coding assistant would, without any persistent engineering knowledge from previous work.

The **experimental condition** consisted of the same repository augmented with EMS. Before each prompt, the agent received engineering knowledge accumulated during the preceding prompts in the form of structured working memory. This memory captured implementation decisions, architectural observations, debugging discoveries, repository-specific assumptions, and evolving engineering beliefs generated throughout the session.

The purpose of this design was not to compare two different implementations or two different engineering strategies. Both agents received the same task and were expected to produce the same software. The only difference was whether one of them was allowed to carry forward the understanding developed during earlier stages of the project.

This distinction is important. The independent variable was **not additional context** in the conventional sense of retrieval-augmented generation or conversation history. Instead, it represented the presence or absence of accumulated engineering cognition - the evolving understanding that EMS attempts to preserve across long-running software engineering sessions.

5.3) Controlled Variables

To ensure that any behavioural differences could reasonably be attributed to engineering cognition rather than unrelated factors, every other aspect of the experiment was intentionally held constant.

Both experimental conditions operated on the **same repository**, beginning from the **same commit** and therefore sharing an identical codebase throughout the evaluation.

The **same language model** was used in both conditions. Model parameters, sampling behaviour, and execution environment remained unchanged so that differences arising from model capability or randomness were minimised.

Each repository received the **same sequence of prompts**, presented in the same order. Every engineering task - from repository exploration through implementation, debugging, testing, and documentation - was therefore identical across both conditions.

Evaluation was also standardised. Every response was assessed using the **same judge prompt**, applying identical scoring criteria across all prompts. The evaluation framework itself never had access to whether a response originated from the EMS-assisted repository or the baseline repository.

Finally, both conditions are executed within the **same local development environment**, using the same operating system, tooling, dependencies, and repository state.

Consequently, the only intentional difference between the two experimental conditions was the presence of EMS-generated engineering cognition. Every observed behavioural difference should therefore be interpreted as the consequence of accumulated engineering understanding rather than differences in model capability, repository state, prompt wording, evaluation procedure, or execution environment.

5.4) Prompt sequence

EC-Bench evaluates engineering behaviour over a long-horizon development workflow rather than through isolated programming tasks. The benchmark therefore consists of a fixed sequence of prompts that collectively simulate the lifecycle of a real software engineering task, beginning with repository familiarisation and ending with final production validation.

Every benchmark run executes the identical prompt sequence in the identical order. The prompts themselves never change between experimental conditions. Only the availability of accumulated engineering cognition differs.

For readability, Table 1 summarises the purpose of each prompt. The complete benchmark prompts are published alongside EC-Bench in the benchmark repository.

Prompt	Phase	Objective	Prompt (abridged)
1	Investigation	Repository orientation	Identify the core FastAPI modules involved in the HTTP request lifecycle.
2	Investigation	Trace request execution	Trace a standard GET request from <code>FastAPI.__call__</code> to response serialization.
3	Investigation	Understand request-handler architecture	Explain the factory pattern used by <code>get_request_handler</code> and distinguish between route-time and request-time state.
4	Investigation	Inventory per-request helpers	Enumerate every helper function created inside the per-request <code>app()</code> coroutine and identify captured state.
5	Investigation	Analyse closure capture semantics	Classify closure variables as route-static or per-request to determine what can be safely hoisted.
6	Investigation	Establish a behavioural baseline	Execute the existing streaming test suite and summarise the behaviour each test protects.
7	Investigation	Understand performance measurement	Examine FastAPI's benchmark framework and evaluate whether streaming performance is currently measured.
8	Investigation	Gather empirical evidence	Design an investigation to determine whether helper closures are recreated on every request and assess the evidence.

9	Architectural Reasoning	Understand existing design rationale	Infer why the current architecture creates helpers per request and evaluate its engineering trade-offs.
10	Architectural Reasoning	Select an implementation strategy	Compare alternative architectures for eliminating repeated helper creation and recommend the best approach.
11	Architectural Reasoning	Map repository integration points	Identify every repository location that must change to support the selected architecture and explain each integration point.
12	Architectural Reasoning	Preserve runtime semantics	Review all request-specific state and explain how the new architecture preserves existing behaviour and avoids regressions.
13	Architectural Reasoning	Document architectural rationale	Write the pull request design section describing the problem, observations, chosen architecture, alternatives, and behavioural guarantees.
14	Architectural Reasoning	Design validation strategy	Plan the complete testing strategy, including existing tests, new benchmarks, execution order, and justification for engineering confidence.
15	Implementation	Implement the core abstraction	Implement the foundational production-quality abstraction for the new architecture without integrating it into request execution.
16	Implementation	Begin incremental integration	Integrate the new abstraction into the safest request-handling path while preserving existing behaviour.
17	Implementation	Integrate a complex streaming path	Extend the abstraction into the most complex streaming execution path while preserving lifecycle management and behavioural equivalence.

18	Implementation	Complete implementation	Finish integrating the abstraction and evaluate additional opportunities for reuse or intentional deferral.
19	Implementation	Clean up implementation	Remove obsolete code, improve documentation, and resolve lint issues to match FastAPI repository standards.
20	Implementation	Add benchmark coverage	Design and implement streaming benchmarks that objectively measure the performance impact of the architectural optimization.
21	Implementation	Validate streaming implementation	Run all streaming test suites (<code>stream</code> , <code>SSE</code> , <code>JSONL</code> , cancellation) and report failures if any.
22	Implementation	Validate non-streaming compatibility	Run application, dependency, and response-model tests to ensure non-streaming paths remain unaffected.
23	Debugging	Debug first behavioural regression	Investigate the first failing test, identify the root cause, and propose the smallest behavioural fix.
24	Debugging	Debug streaming lifecycle regression	Investigate a streaming regression by tracing lifecycle, control flow, cleanup, and state management.
25	Debugging	Resolve static-analysis regressions	Run the repository's type-checking workflow, fix new type errors, and preserve behaviour.
26	Debugging	Achieve repository lint compliance	Run Ruff, resolve lint/style issues, and ensure the code matches FastAPI conventions.
27	Debugging	Perform final integration validation	Execute the complete streaming integration test suite and verify that benchmark files skip cleanly without crashing.
28	Planning	Evaluate architectural	Assess whether the new

		generalisation	architecture should also be applied to the WebSocket execution pipeline.
29	Planning	Identify the next optimisation target	Propose the next meaningful performance improvement in the request-handling subsystem.
30	Planning	Recommend long-term architectural direction	Propose a broader architectural improvement for the request execution pipeline based on lessons learned.

5.5) Evaluation Metrics

Traditional software engineering benchmarks primarily evaluate whether an agent eventually produces correct code. While useful, such benchmarks provide limited insight into whether an agent's engineering behaviour improves as understanding accumulates over time.

EC-Bench therefore evaluates engineering behaviour rather than final program correctness alone. The benchmark measures multiple complementary dimensions that together capture whether accumulated engineering cognition influences how software engineering is performed across a long-running development session.

The evaluation metrics were designed around the central hypothesis of this work: if engineering cognition exists, later engineering decisions should increasingly reflect previously accumulated understanding rather than repeated rediscovery.

The current version of EC-Bench evaluates the following dimensions:

- **Architectural Continuity** — whether architectural decisions remain coherent as implementation progresses.
- **Repository Groundedness** — whether engineering decisions are rooted in the actual repository rather than in generic software knowledge.
- **Engineering Cognition Reuse** — whether previously accumulated engineering understanding actively influences future engineering decisions.
- **Engineering Quality** — the intrinsic quality of the engineering solution independent of cognition.
- **Debugging & Investigation Efficiency** — how efficiently the agent converges toward root causes during debugging and investigation.

Each metric is independently scored on a continuous scale from 0 to 10. The benchmark intentionally separates these dimensions because improvements in one engineering behaviour

do not necessarily imply improvements in another. For example, an implementation may exhibit excellent engineering quality while repeatedly reconstructing context that should already have been available through accumulated cognition.

Weighted aggregation is used only to provide a high-level summary score. Individual metric behaviour remains the primary object of analysis throughout this paper.

A complete formal specification of every evaluation metric, including scoring guidelines and weighting strategy, is published with the EC-Bench benchmark.

5.6) Judge

Every EC-Bench response is evaluated independently by an LLM-based Judge Agent specifically designed for engineering evaluation.

The Judge does not generate solutions, propose improvements, or compare responses directly. Its sole responsibility is to evaluate the engineering behaviour demonstrated by a completed response.

To minimise evaluation bias, the Judge follows several design principles.

First, the Judge is **blind to the experimental condition**. It is never informed whether a response originated from the baseline repository or the repository augmented with the Engineering Memory System (EMS). Each response is evaluated in isolation against the current repository state.

Second, the Judge follows a **repository-first evaluation strategy**. Before evaluating a response, it reconstructs a sufficient understanding of the repository architecture, conventions, abstractions, testing strategy, and dependency structure. Repository implementation always takes precedence over accumulated engineering cognition whenever disagreements arise.

Third, the Judge evaluates responses across the predefined EC-Bench metrics rather than relying on an overall subjective impression. Every metric is scored independently, requiring explicit engineering evidence before credit can be assigned. Longer responses, more confident writing, or unnecessary architectural complexity are never rewarded.

Finally, the Judge is instructed to explain every assigned score using concrete engineering evidence drawn from the repository and the submitted response. This produces evaluations that remain interpretable while making it possible to understand why individual scores change across benchmark runs.

Although LLM-based evaluation inevitably introduces some subjectivity, we believe this approach currently provides the best trade-off between scalability and engineering-aware assessment. Importantly, the Judge prompt, evaluation procedure, and scoring guidelines are fully published as part of EC-Bench to maximise transparency and reproducibility.

5.7) Results

5.7.1) Overall Performance

5.7.1.1) Overall score table:

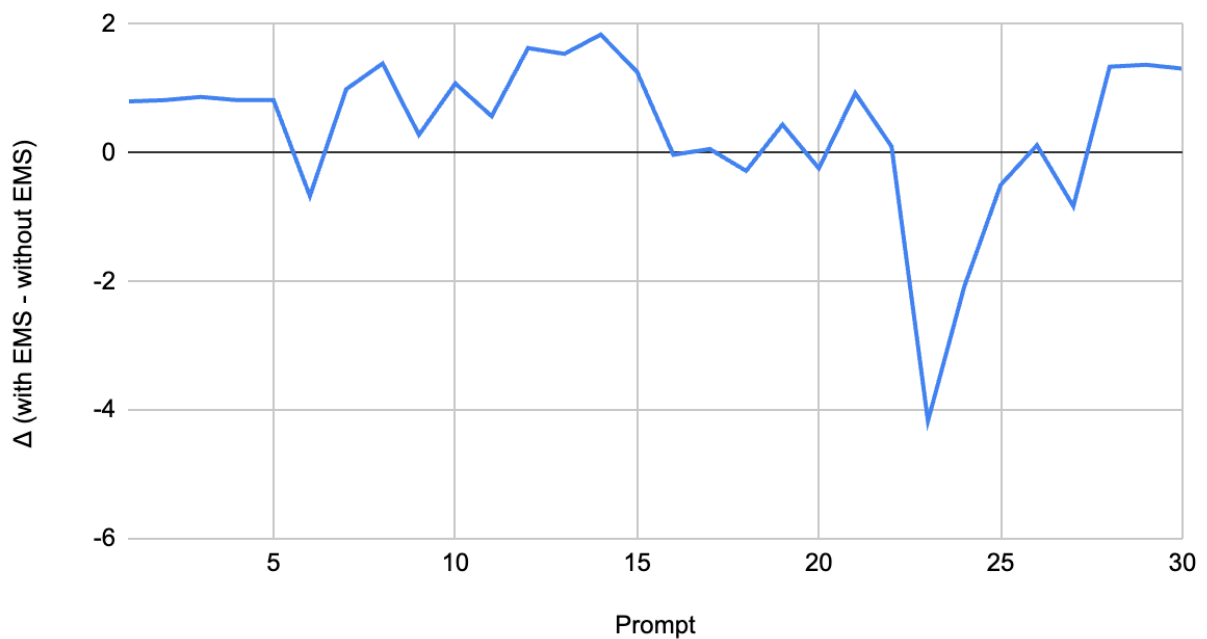
Prompt	With EMS	Without EMS	Δ (with EMS - without EMS)
1	8.23	7.43	+0.80
2	8.80	7.98	+0.82
3	8.95	8.08	+0.87
4	8.60	7.78	+0.82
5	8.77	7.95	+0.82
6	7.08	7.75	-0.67
7	8.67	7.68	+0.99
8	9.17	7.78	+1.39
9	8.51	8.23	+0.28
10	8.91	7.83	+1.08
11	8.72	8.15	+0.57
12	9.38	7.75	+1.63
13	9.09	7.55	+1.54
14	9.19	7.35	+1.84
15	8.81	7.55	+1.26
16	8.77	8.80	-0.03
17	9.11	9.05	+0.06
18	8.83	9.11	-0.28
19	8.36	7.92	+0.44
20	8.61	8.85	-0.24
21	8.28	7.35	+0.93

22	7.45	7.35	+0.10
23	4.68	8.85	-4.17
24	6.10	8.18	-2.08
25	7.73	8.23	-0.50
26	8.20	8.08	+0.12
27	6.35	7.18	-0.83
28	8.92	7.58	+1.34
29	9.17	7.80	+1.37
30	9.11	7.80	+1.31

Key: Positive deltas(marked green) indicate that EMS improved overall engineering performance relative to a stateless coding agent. Negative deltas(marked red) indicate prompts where accumulated engineering cognition either failed to improve performance or introduced new failure modes. These prompts are analysed individually in Section 7.

5.7.1.2) Overall score line chart:

Δ (with EMS - without EMS) vs. Prompt

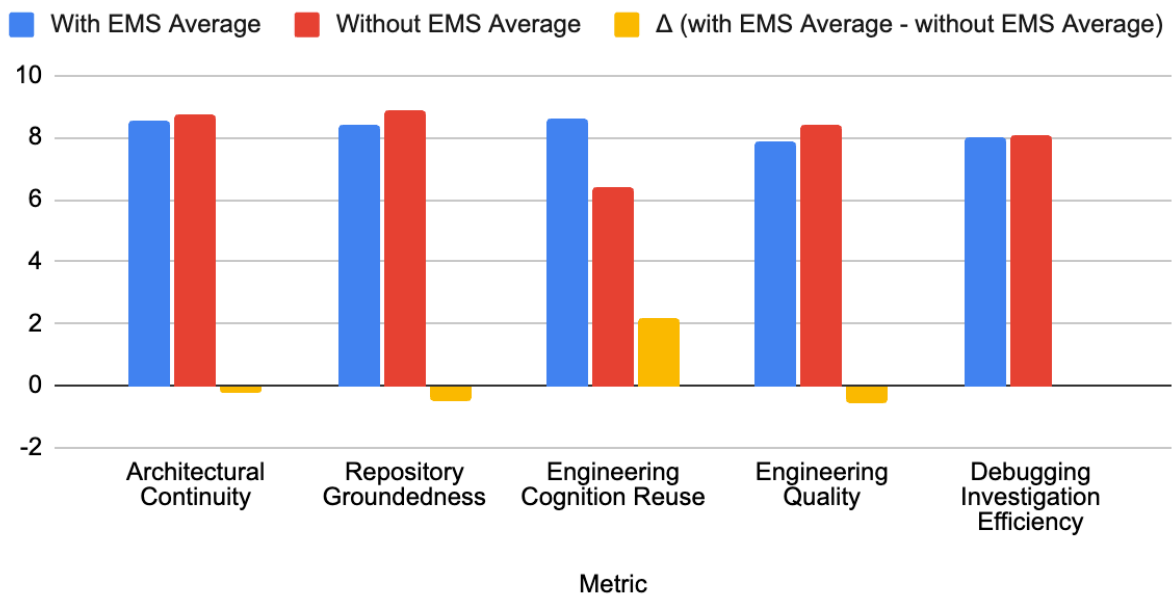


5.7.2) Metric-wise Performance

5.7.2.1) Average score table

Metric	With EMS Average	Without EMS Average	Δ (with EMS Average - without EMS Average)
Architectural Continuity	8.543	8.766	-0.223
Repository Groundedness	8.396	8.876	-0.480
Engineering Cognition Reuse	8.590	6.376	+2.214
Engineering Quality	7.844	8.433	-0.589
Debugging Investigation Efficiency	7.993	8.050	-0.057

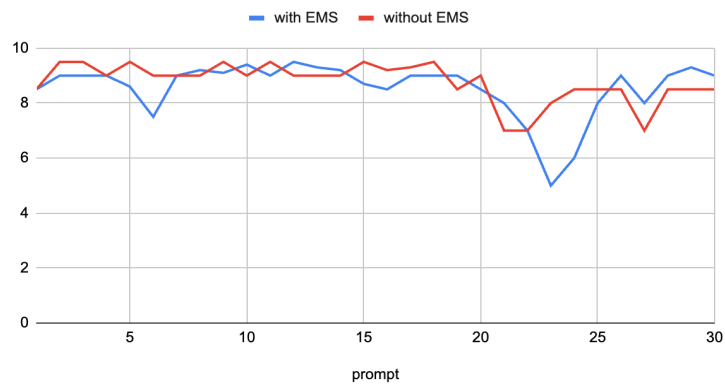
With EMS Average, Without EMS Average and Δ (with EMS Average - without EMS Average)



5.7.2.2) Five metric trend chart

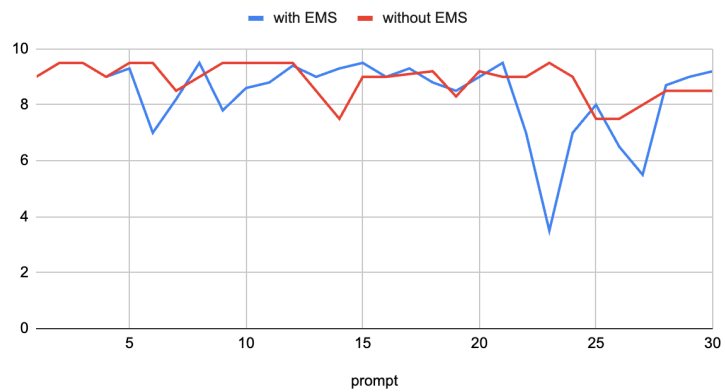
A) Architectural Continuity

with EMS and without EMS



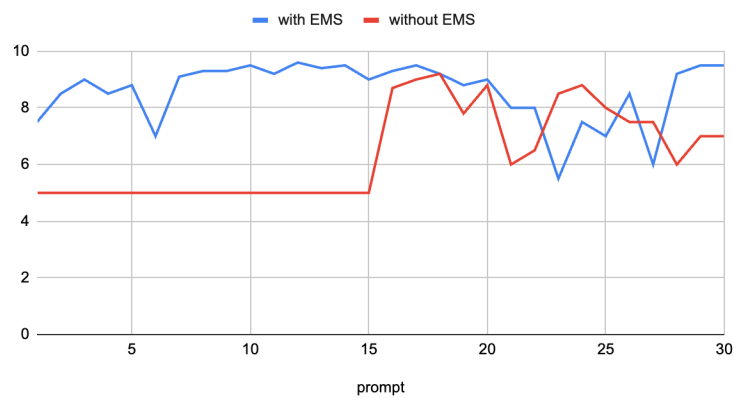
B) Repository Groundedness

with EMS and without EMS



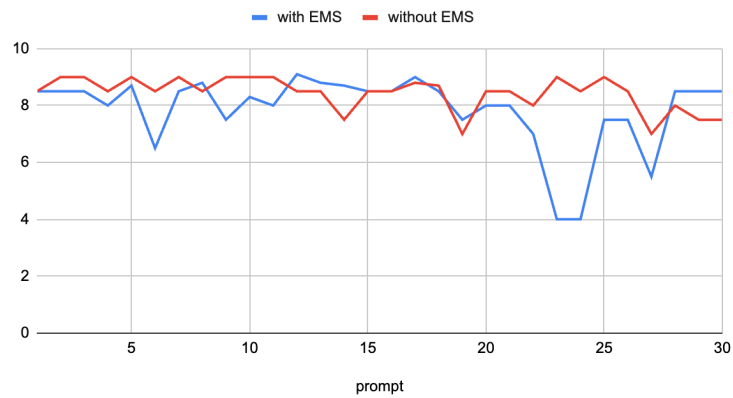
C) Engineering Cognition Reuse

with EMS and without EMS



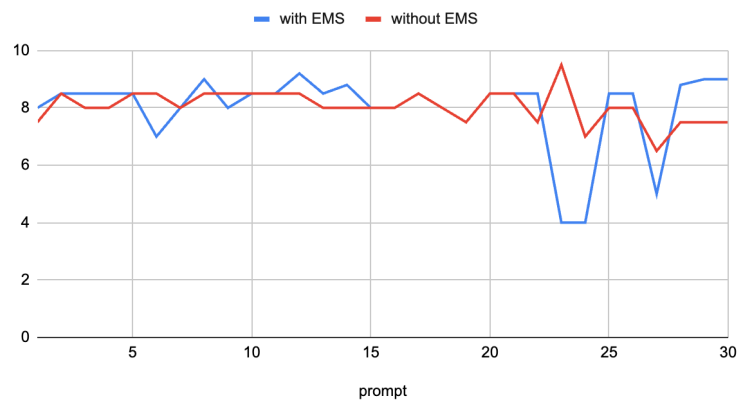
D) Engineering Quality

with EMS and without EMS



E) Debugging Investigation Efficiency

with EMS and without EMS



5.7.3) Summary Statistics

5.7.3.1) Overall Outcome

Statistic	Value
Total prompts	30
EMS wins	22
Stateless wins	8
Mean score (with EMS)	8.35

Mean score (without EMS)	7.96
Mean improvement	+0.37

Across the complete benchmark, the Engineering Memory System outperformed the stateless baseline on 22 of the 30 benchmark prompts, with an average improvement of **0.368 points** per prompt. Although the aggregate result is positive, the improvement is not uniformly distributed across the benchmark. Several prompts exhibit substantial gains, while a small number of debugging-oriented prompts produce large regressions that dominate the negative tail of the distribution.

5.7.3.2) Largest Improvements

Prompt	Δ Score	Phase	Why it helped
14	+1.84	Architectural Reasoning	Existing engineering context enabled a comprehensive validation strategy instead of reconstructing one from scratch.
12	+1.63	Architectural Reasoning	Previously accumulated understanding of request-specific state was directly reused.
13	+1.54	Architectural Reasoning	Existing design rationale enabled the PR design document to be produced with greater consistency.
8	+1.39	Investigation	Prior repository understanding allowed the investigation to focus immediately on hypothesis validation rather than repository exploration.
29	+1.37	Planning	Previously accumulated

			subsystem understanding enabled a stronger forward-looking engineering proposal.
--	--	--	--

A clear pattern emerges from the highest-scoring prompts. The largest improvements consistently occur during investigation, architectural reasoning and engineering planning tasks. These phases require the agent to synthesise information accumulated over multiple earlier interactions rather than rediscover repository knowledge. Engineering Cognition, therefore, appears to provide the greatest benefit when the task requires long-horizon reasoning instead of immediate implementation.

5.7.3.3) Largest Regressions

Prompt	Δ Score	Phase	Observed issue
23	-4.17	Debugging	Retrieved engineering context biased the investigation before sufficient repository evidence had been gathered.
24	-2.08	Debugging	Previous architectural assumptions influenced debugging earlier than repository observations.
27	-0.83	Debugging	Prior context encouraged workaround-oriented reasoning rather than strict execution of the requested verification workflow.
6	-0.67	Investigation	Existing context slightly reduced repository verification during baseline testing.
25	-0.50	Debugging	Retrieved knowledge

			influenced the debugging process before static-analysis evidence had been fully established.
--	--	--	--

Unlike the positive results, the largest regressions are concentrated almost exclusively within debugging tasks. In these prompts, previously accumulated engineering understanding occasionally influenced the direction of the investigation before sufficient repository evidence had been collected. Rather than beginning from the newly observed failure, the agent sometimes attempted to explain the failure using existing engineering beliefs. This suggests that the current Engineering Memory System encourages retrieval before observation during debugging workflows. The remainder of this report investigates this behaviour in greater detail and proposes architectural changes intended to address it.

5.7.3.4) Metric Summary

Examination of the individual evaluation metrics reveals an unexpected outcome. Engineering Cognition substantially improves the **Engineering Cognition Reuse** metric (+2.214), demonstrating that the accumulated engineering context is actively incorporated into subsequent reasoning. However, every remaining metric shows a modest decline relative to the stateless baseline. The largest reductions occur in **Engineering Quality** (−0.589) and **Repository Groundedness** (−0.480), suggesting that although the retrieved engineering context is reused successfully, it does not consistently improve engineering performance. This finding became the central motivation for redesigning the Engineering Memory System following the completion of this experiment.

Observation 1

Engineering Cognition demonstrably changes the reasoning process of the coding agent. However, under the current EMS architecture, increased reuse of engineering understanding does not consistently translate into higher engineering performance. Understanding this discrepancy became the primary outcome of the first EC-Bench experiment.

6) Prompt-Level findings

6.1) Investigation prompts

Prompt 6

Objective

Verify the current behaviour of FastAPI's streaming endpoints before beginning the refactor by executing the existing streaming test suite and identifying exactly what behaviour each test protects.

Outcome

	With EMS	Without EMS	Δ
Overall Score	7.08	7.75	-0.67

Observation

Both systems successfully executed the streaming test suite and correctly established that all tests passed. The stateless system accurately mapped every test to the corresponding implementation behaviour, whereas EMS incorrectly identified the execution path exercised by the bare `AsyncIterable` streaming test. Rather than introducing an incorrect overall understanding, EMS reused previously accumulated architectural knowledge without re-verifying one repository-specific implementation detail.

Key Insight

This prompt revealed the first important limitation of the current EMS architecture. Engineering cognition successfully preserved the high-level architectural model of the streaming pipeline, but repository-specific implementation facts were treated with the same confidence as verified observations. The result was not an architectural misunderstanding, but an unverified engineering memory that propagated into the final answer.

Prompt-specific Lesson

- Engineering memories should carry confidence levels, and low-confidence implementation facts should automatically trigger repository verification before being reused.

Planned Changes

EMS

- Introduce confidence levels for stored engineering memories.
- Record provenance for every extracted engineering artifact.
- Require automatic verification before reusing correctness-critical implementation details.

Judge

- Separate repository-groundedness errors from architectural reasoning errors.
- Distinguish verified memory reuse from unverified memory reuse during evaluation.
- Reward verification of remembered knowledge rather than reuse alone.

EC-Bench

- Add prompts that intentionally contain repository facts requiring verification.
- Include tasks where architectural understanding is correct but implementation details have changed.
- Evaluate whether the agent knows when to verify accumulated engineering cognition.

Prompt 8

Objective

Empirically investigate whether helper functions defined inside FastAPI's per-request request handler are recreated on every incoming request and determine whether this behaviour introduces measurable runtime overhead.

Outcome

	With EMS	Without EMS	Δ
Overall Score	9.17	7.78	+1.39

Observation

Both systems concluded that nested helper functions are recreated on every request while their underlying bytecode objects remain shared. However, EMS designed and executed a substantially richer investigation by instrumenting the actual FastAPI implementation, inspecting closure identities, differentiating route-static and request-specific closure cells, and validating these observations through runtime experimentation. In contrast, the stateless system relied primarily on simplified proxy experiments and architectural inference.

Key Insight

Engineering cognition significantly improved the quality of experimental design. Because EMS retained the architectural understanding accumulated during earlier investigation prompts—including the closure factory structure, free-variable analysis, and route-static versus request-specific variable classification—it was able to formulate a much more targeted

experiment. Rather than merely confirming the original hypothesis, the investigation produced a deeper engineering result by demonstrating that only request-specific closure cells are recreated while route-static cells remain shared across requests.

Prompt-specific Lesson

- Accumulated engineering cognition enables progressively more sophisticated investigations, allowing later experiments to validate previous architectural reasoning rather than rediscover it from scratch.

Planned Changes

EMS

- Continue preserving intermediate architectural discoveries as reusable engineering cognition.
- Improve extraction so experimentally verified observations receive higher confidence than inferred conclusions.
- Automatically promote validated architectural findings into long-term engineering memory.

Judge

- Explicitly reward investigations that extend previously accumulated engineering understanding.
- Differentiate between hypothesis confirmation and discovery of new architectural behaviour.
- Increase emphasis on empirical validation performed on the actual repository.

EC-Bench

- Include investigation prompts that naturally build upon earlier architectural discoveries.
- Reward cumulative experimental reasoning instead of isolated one-shot investigations.
- Expand benchmark coverage for instrumentation, profiling, and empirical architecture validation.

6.2) Architectural reasoning prompts

Prompt 12

Objective

Evaluate whether the proposed architecture preserves existing runtime behaviour by correctly propagating request-specific state through the new design.

Outcome

	With EMS	Without EMS	Δ
Overall Score	9.38	7.75	+1.63

Observation

Both models successfully identified the request-specific state involved in the streaming pipeline and correctly classified most variables as either request-specific or route-static. The repository-grounded analysis remained equally strong in both cases. The primary difference was that the EMS-assisted model directly extended the previously selected architecture, whereas the baseline reconstructed that architecture from the repository before analysing state propagation.

Key Insight

This prompt required architectural continuity rather than additional repository exploration. EMS allowed the model to reuse previously established decisions about helper hoisting, closure behaviour, and integration boundaries, enabling it to reason directly about runtime semantics. Without persistent engineering memory, the model spent a significant portion of its reasoning budget rediscovering assumptions that had already been established in earlier prompts.

Prompt-specific Lesson

- Engineering cognition should become the primary source of architectural context once the implementation strategy has been established.

Planned Changes

- **EMS:** Store explicit architectural decisions (selected design, rejected alternatives, behavioural guarantees) as first-class engineering memory.

- **Judge:** Reward reuse of previously established architectural decisions instead of rewarding repeated repository rediscovery.
- **EC-Bench:** Introduce more prompts requiring preservation of behavioural semantics across multiple architectural reasoning stages.

Prompt 13

Objective

Produce a concise pull request Design section that accurately summarises the engineering problem, architectural decision, alternatives, and behavioural guarantees.

Outcome

	With EMS	Without EMS	Δ
Overall Score	9.09	7.55	+1.54

Observation

Both responses produced well-structured design documents that matched the expected FastAPI maintainer style. However, the EMS-assisted response synthesised conclusions accumulated across the previous architectural prompts into a coherent engineering narrative. The baseline produced a technically correct document, but reconstructed much of the rationale directly from repository inspection instead of reusing previously derived reasoning.

Key Insight

This prompt contained almost no new repository discovery. Its difficulty lay entirely in summarising the accumulated engineering investigation into a concise, reviewable document. EMS transformed the task from repeated reasoning into knowledge synthesis, allowing the model to compress prior architectural analysis into a consistent pull request design.

Prompt-specific Lesson

- Engineering cognition should be reusable for engineering communication tasks, where repository search provides little additional value.

Planned Changes

- **EMS:** Generate structured design summaries automatically after major architectural reasoning phases.
- **Judge:** Explicitly reward consistency between the final design document and architectural decisions made in previous prompts.
- **EC-Bench:** Add more engineering communication prompts (design documents, RFCs, PR descriptions, ADRs) that depend primarily on accumulated engineering cognition rather than on repository search.

Prompt 14:

Objective

Validate that the proposed implementation preserves existing behaviour by reviewing the final code as if performing a post-implementation engineering review.

Outcome

	With EMS	Without EMS	Δ
Overall Score	9.19	7.35	+1.84

Observation

Both systems produced technically correct post-implementation reviews and correctly identified the implementation as preserving runtime behaviour. The EMS version, however, remained tightly aligned with the engineering decisions accumulated across the previous architectural prompts, whereas the baseline reconstructed the same reasoning directly from repository inspection. The largest difference came from Engineering Cognition Reuse, where EMS demonstrated continuity with previously established architectural decisions instead of rediscovering them.

Key Insight

Prompt 14 measures whether engineering reasoning remains consistent after implementation is complete. Unlike Prompt 12 (state analysis) or Prompt 13 (design justification), this task benefits directly from accumulated engineering cognition because the implementation must be evaluated against earlier design decisions rather than treated as an isolated code change. EMS therefore improved consistency and reduced rediscovery, although repository verification remained necessary to confirm the final implementation.

Prompt-specific Lesson

- Engineering cognition is most valuable during post-implementation validation, where the implementation must be evaluated against previously established architectural decisions rather than rediscovered from repository inspection.

Planned Changes

- **EMS:** Retrieve implementation decisions and behavioural guarantees as a dedicated "decision history" artefact during review prompts instead of replaying the broader architectural context.
- **Judge:** Reward consistency between the implementation review and previously established architectural decisions as a distinct aspect of Engineering Cognition Reuse.
- **EC-Bench:** Add additional implementation-review prompts that require validating completed code against earlier architectural commitments instead of evaluating the implementation in isolation.

6.3) Implementation prompts

Prompt 15:

Objective

Implement the first production-quality architectural abstraction established during the design phase without integrating it into the request execution path.

Outcome

	With EMS	Without EMS	Δ
Overall Score	8.81	7.55	+1.26

Observation

Both systems implemented the requested abstraction while preserving existing behaviour. The EMS version produced the complete production-quality component anticipated by earlier architectural work, including the full family of streaming serialisation helpers, whereas the baseline implemented only the first helper and deferred the remaining abstraction to later prompts. The largest performance gain came from Engineering Cognition Reuse, where EMS directly reused previously established implementation plans instead of reconstructing the architectural decomposition from scratch.

Key Insight

This prompt measures whether an implementation can continue an architectural plan that has already been developed across multiple previous prompts. EMS did not simply remember architectural decisions; it reused the previously established implementation decomposition, integration boundaries, and validation strategy to construct the complete foundational abstraction immediately. The baseline remained technically correct but treated the implementation as an isolated task, leading to a more incremental and incomplete realisation of the architecture.

Prompt-specific Lesson

- Engineering cognition is most valuable when implementation requires continuing an existing architectural decomposition rather than independently deciding how to structure the new abstraction.

Planned Changes

- **EMS:** Store implementation decomposition (planned components, sequencing, and integration boundaries) as a first-class engineering cognition artefact instead of only preserving architectural decisions.
- **Judge:** Reward implementations that correctly continue previously established implementation plans rather than evaluating completeness purely within the current prompt.
- **EC-Bench:** Add implementation prompts that require realising architectural decompositions established across multiple earlier prompts instead of implementing isolated components.

Prompt 20:

Objective

Extend benchmark coverage so the engineering motivation behind the implementation can be evaluated objectively.

Outcome

	With EMS	Without EMS	Δ
Overall Score	8.61	8.85	-0.24

Observation

Both systems designed appropriate benchmark coverage for the streaming refactor and correctly identified the missing streaming benchmarks in the existing suite. The baseline achieved a slightly higher score because it remained tightly aligned with the repository's existing benchmark architecture, whereas the EMS version expanded the benchmark suite more aggressively by introducing additional benchmark dimensions and allocation diagnostics that extended beyond the repository's established conventions. The largest regressions occurred in Architectural Continuity and Engineering Quality rather than Engineering Cognition Reuse.

Key Insight

Prompt 20 demonstrates that benchmark design depends more heavily on existing repository conventions than on accumulated engineering cognition. Although EMS successfully reused prior knowledge about expected performance characteristics and benchmark objectives, this additional context encouraged broader benchmark coverage that partially diverged from the repository's established benchmarking style. In implementation-oriented tasks that primarily

extend existing infrastructure, repository grounding remains a stronger signal than accumulated engineering context.

Prompt-specific Lesson

- Engineering cognition should complement repository conventions during benchmark design rather than expanding benchmark scope beyond the engineering patterns already established within the repository.

Planned Changes

- **EMS:** Prioritise repository benchmark conventions before incorporating accumulated engineering context when generating benchmark implementations.
- **Judge:** Penalise benchmark designs that unnecessarily extend beyond the repository's existing benchmarking methodology, even when technically justified.
- **EC-Bench:** Add benchmark-extension prompts that distinguish between correctly following repository benchmarking conventions and unnecessarily expanding benchmark scope.

6.4) Debugging prompts

Prompt 23

Objective

Investigate the first behavioural regression introduced by the implementation and identify the engineering reasoning required to isolate its root cause.

Outcome

	With EMS	Without EMS	Δ
Overall Score	4.68	8.85	-4.17

Observation

The baseline substantially outperformed the EMS-assisted system because it remained grounded in the actual repository and accurately identified the regression as an environmental issue rather than an implementation bug. The EMS version, instead, constructed an entirely hypothetical failure scenario and spent most of the investigation debugging a callback architecture that did not exist in the implementation.

The largest degradation occurred in Repository Groundedness and Debugging Investigation Efficiency. Although EMS successfully recalled the canonical architectural decision about hoisting serialisation helpers and attempted to reuse previous engineering context, that

accumulated knowledge was incorrectly applied because the investigation never verified the current implementation before forming hypotheses. As a result, the entire debugging process diverged from the repository and ultimately proposed a fix for code that did not exist.

Key Insight

Prompt 23 demonstrates that debugging investigations require repository validation before accumulated engineering cognition is reused. Architectural memory is only beneficial after the current implementation has been confirmed. Otherwise, previously accumulated reasoning can bias the investigation toward imagined implementation details instead of actual code.

Prompt-specific Lesson

Engineering cognition should guide debugging only after the current implementation has been verified. Repository inspection must precede architectural reasoning, particularly when investigating regressions where the implementation may differ from previously accumulated assumptions.

Planned Changes

- **EMS:** Introduce a mandatory repository verification phase before forming debugging hypotheses. Require the investigation to first inspect the relevant implementation before consulting the accumulated engineering memory.
- **Judge:** Penalise debugging investigations that construct hypothetical execution paths without first validating the current implementation against the repository.
- **EC-Bench:** Add debugging prompts that explicitly distinguish between repository-grounded investigations and investigations driven primarily by previously accumulated architectural knowledge.

Prompt 24

Objective

Investigate a regression affecting the streaming execution lifecycle and determine whether the behaviour changes because of lifecycle management, resource cleanup, concurrency, control flow, or state management.

Outcome

	With EMS	Without EMS	Δ
Overall Score	6.10	8.18	-2.08

Observation

Both systems correctly traced the streaming execution lifecycle and recognised that the implementation largely preserved the original streaming behaviour. However, the baseline investigation remained concise, consistently grounded in the repository, and concluded that no behavioural regression existed after systematically analysing each streaming path.

The EMS-assisted investigation showed a clear improvement over Prompt 23 by abandoning the earlier hypothetical callback architecture and correctly reading the repository. It successfully reused accumulated engineering understanding regarding explicit parameter passing, streaming architecture, and the canonical optimisation. Nevertheless, the investigation repeatedly revisited the same hypotheses, eventually contradicting its own earlier conclusions by proposing a lifecycle regression that it had previously demonstrated did not exist. This inconsistency significantly reduced both Engineering Quality and Debugging Investigation Efficiency.

Key Insight

Prompt 24 demonstrates that engineering cognition becomes substantially more valuable once repository grounding has already been established. EMS successfully leveraged accumulated architectural understanding to navigate a complex streaming lifecycle. However, debugging quality depends equally on reasoning discipline. Repeatedly reconsidering already-resolved hypotheses introduced contradictions that ultimately weakened an otherwise repository-grounded investigation.

Prompt-specific Lesson

Engineering cognition should accelerate lifecycle reasoning after repository verification, but once a hypothesis has been conclusively eliminated, it should not be reopened without new evidence. Stable reasoning is as important as accumulated engineering knowledge during complex debugging investigations.

Planned Changes

- **EMS:** Add an explicit hypothesis tracking stage that records when a hypothesis has been confirmed or eliminated, preventing the investigation from repeatedly revisiting previously resolved reasoning.
- **Judge:** Penalise investigations that contradict their own established conclusions without introducing new supporting evidence.
- **EC-Bench:** Introduce lifecycle debugging prompts that reward stable hypothesis elimination and consistent reasoning across long multi-stage investigations.

Prompt 25

Objective

Resolve any static-analysis regressions introduced by the implementation while preserving the intended architecture and runtime behaviour.

Outcome

	With EMS	Without EMS	Δ
Overall Score	7.73	8.23	-0.50

Observation

Both systems correctly recognised that the implementation itself remained behaviourally stable and focused exclusively on resolving static-analysis regressions.

The baseline investigation remained tightly scoped, identifying only the regression directly introduced by the recent optimisation and resolving it with the smallest possible change. Throughout the investigation it stayed closely aligned with the repository's existing engineering conventions while clearly distinguishing implementation regressions from pre-existing issues.

The EMS-assisted investigation performed a much broader static-analysis review and systematically resolved a larger set of typing issues. Although the individual fixes were technically correct and the investigation was thorough, it gradually expanded beyond the scope of the implementation by treating existing repository issues as regressions introduced by the optimisation. As a result, EMS demonstrated stronger debugging coverage but weaker adherence to the repository's established engineering practices, reducing both Repository Groundedness and Engineering Quality.

Key Insight

Prompt 25 shows that engineering cognition naturally encourages broader repository-wide investigations. While this increases thoroughness, it also introduces a tendency to "improve" nearby code that was never part of the implementation under evaluation. Static-analysis debugging therefore benefits not only from identifying errors but also from accurately distinguishing between **new regressions**, **pre-existing debt**, and **intentional repository conventions**.

Prompt-specific Lesson

Accumulated engineering knowledge should expand the investigation scope only after the

implementation's own regressions have been isolated. Correctly fixing unrelated type issues is valuable engineering work, but those fixes should not be attributed to the implementation being evaluated.

Planned Changes

- **EMS:** Introduce a regression classification stage that explicitly separates *implementation-introduced errors*, *pre-existing repository issues*, and *optional repository improvements* before proposing fixes.
- **Judge:** Increase penalties when investigations incorrectly classify existing repository debt as newly introduced regressions.
- **EC-Bench:** Include static-analysis tasks containing deliberate pre-existing typing debt to evaluate whether agents can accurately isolate implementation regressions from unrelated repository issues.

Prompt 27

Objective

Execute the final integration validation by running the complete streaming-related test suite with fail-fast, verify benchmark behaviour without `--codspeed`, and confirm that the implementation is production-ready.

Outcome

	With EMS	Without EMS	Δ
Overall Score	6.35	7.18	-0.83

Observation

Both systems successfully demonstrated that the streaming implementation itself was stable. They verified that the relevant streaming tests passed and that the benchmark suite behaved correctly when the benchmarking framework was unavailable.

The primary difference lay in how each system interpreted the requested validation procedure.

The baseline investigation remained transparent about the limitations imposed by unrelated repository environment issues. Rather than conflating these external problems with implementation correctness, it explicitly distinguished between environmental failures and feature validation, explaining why an alternative verification strategy was used while clearly communicating the remaining limitations.

The EMS-assisted investigation instead broadened the scope of the task by attempting to repair repository-level infrastructure before completing validation. While these additional changes were technically reasonable, they were not required by the prompt. More importantly, EMS ultimately reported complete validation despite the original verification procedure remaining only partially executable, leading to an overstatement of implementation confidence.

Key Insight

Prompt 27 illustrates that engineering cognition should improve confidence in implementation correctness, not broaden the definition of success. Once unrelated infrastructure problems are encountered, the investigation must clearly distinguish **implementation validation** from **repository health**. Expanding the scope to repair an unrelated configuration can obscure whether the requested validation was actually completed.

Prompt-specific Lesson

Final integration validation should prioritise faithful execution of the requested verification workflow. When environmental issues prevent exact execution, the investigation should explicitly report the limitation rather than implicitly substituting a narrower validation and presenting it as equivalent.

Planned Changes

- **EMS:** Add a verification integrity stage that records whether the exact requested validation was completed, partially completed, or substituted with an alternative workflow, preventing over-reporting of final confidence.
- **Judge:** More heavily penalise investigations that report complete validation after silently replacing the requested verification procedure with a narrower alternative.
- **EC-Bench:** Introduce integration prompts containing unrelated repository collection failures, requiring agents to distinguish implementation correctness from environment-level issues while accurately reporting validation coverage.

6.5) Planning prompts

Prompt 29

Objective

Review the surrounding implementation after completing the optimisation and identify the next meaningful performance improvement within the same subsystem by proposing a follow-up engineering task.

Outcome

	With EMS	Without EMS	Δ
Overall Score	9.17	7.80	+1.37

Observation

Both systems proposed technically sound follow-up engineering work that naturally extended the recently completed optimisation.

The baseline proposal identified another opportunity elsewhere in the surrounding implementation where the same optimisation principle could be applied. The proposal remained well grounded in the repository and demonstrated strong architectural continuity, but it primarily relied on analysing the current implementation to discover another independent optimisation opportunity.

The EMS-assisted proposal instead treated the completed optimisation as an ongoing engineering effort. Rather than searching for a new optimisation elsewhere in the repository, it recognised that the existing optimisation remained incomplete and proposed the next logical step required to finish the work already underway. This demonstrated substantially stronger Engineering Cognition Reuse by leveraging accumulated implementation knowledge to preserve optimisation intent across planning tasks, rather than restarting architectural discovery from the current repository state.

Key Insight

Prompt 29 demonstrates that engineering cognition is particularly valuable during planning tasks. Once implementation knowledge has accumulated, follow-up work no longer needs to be rediscovered through repository inspection alone. Instead, previously established architectural principles can guide the identification of unfinished optimisation opportunities that naturally extend existing engineering work.

Prompt-specific Lesson

Planning quality improves when accumulated engineering knowledge preserves implementation intent rather than only implementation facts. Engineering memory enables future work to evolve existing optimisation programmes incrementally instead of repeatedly restarting architectural discovery from the repository.

Planned Changes

- **EMS:** Explicitly maintain optimisation lineage by recording incomplete optimisation patterns and remaining opportunities within the same engineering area, allowing future planning tasks to continue existing work before proposing unrelated improvements.
- **Judge:** Increase emphasis on recognising whether proposed follow-up work naturally extends the implementation that has just been completed, rather than simply identifying another valid optimisation elsewhere in the repository.
- **EC-Bench:** Introduce planning prompts that distinguish between discovering independent optimisation opportunities and continuing an existing optimisation programme, rewarding proposals that demonstrate long-horizon engineering continuity rather than isolated architectural analysis.

7) Cross-Prompt Findings

The prompt-level analyses revealed several recurring behavioural patterns that extended beyond individual tasks. Rather than representing isolated successes or failures, these patterns exposed fundamental properties of how engineering cognition should be accumulated, retrieved, and applied during long-horizon software engineering. The following findings directly informed the subsequent redesign of both EMS and EC-Bench.

7.1) Retrieval Behaviour

One of the strongest findings from this experiment is that **engineering knowledge should not be retrieved eagerly**. Across multiple prompts, we observed that retrieving accumulated knowledge before first understanding the current repository frequently caused the investigation to anchor on outdated assumptions or previously established beliefs. In contrast, the highest-quality investigations consistently verified the repository first and retrieved additional engineering knowledge only when it became necessary to resolve uncertainty.

This suggests that engineering cognition should be **demand-driven rather than context-driven**. Instead of maximising the amount of retrieved information, future engineering memory systems should optimise **retrieval timing**, allowing repository evidence to establish the initial understanding before accumulated engineering knowledge is introduced.

7.2) Scope Expansion

A second recurring pattern was that **engineering cognition naturally expands problem scope**. As more engineering understanding accumulated, EMS increasingly identified opportunities beyond the immediate task, proposing repository improvements, follow-up optimisations, and architectural refinements that were not explicitly requested.

This behaviour proved to be both beneficial and risky. During planning tasks, scope expansion enabled stronger long-term engineering proposals by extending existing implementation work instead of repeatedly rediscovering new optimisation opportunities. During debugging and verification tasks, however, the same behaviour sometimes caused investigations to solve adjacent engineering problems instead of completing the requested objective. These results suggest that future engineering cognition systems should treat **scope expansion as an explicit reasoning decision**, rather than an automatic consequence of accumulated knowledge.

7.3) Engineering Modes

A third finding is that **different phases of software engineering require fundamentally different forms of engineering cognition**. Investigation, architectural reasoning, implementation, debugging, and planning each benefited from different patterns of knowledge

reuse and reasoning. No single retrieval strategy consistently improved performance across every engineering activity.

This suggests that engineering cognition should become **mode-aware**. Future systems should adapt how knowledge is retrieved, accumulated, and applied according to the engineering task being performed, rather than treating all software engineering activities as requiring the same cognitive process.

7.4) What Actually Improved?

The experiment also demonstrates that **engineering cognition does not uniformly improve software engineering performance**. The largest improvements consistently appeared in architectural reasoning and long-term planning, where accumulated engineering understanding enabled the system to extend previous work instead of repeatedly rediscovering it.

In contrast, early investigation and debugging benefited far less consistently. These phases depended heavily on repository verification and disciplined hypothesis management, meaning that accumulated engineering knowledge could occasionally interfere with reasoning when introduced prematurely. Together, these findings suggest that the primary value of engineering cognition lies not in replacing repository analysis, but in extending engineering understanding once the current implementation has been correctly established.

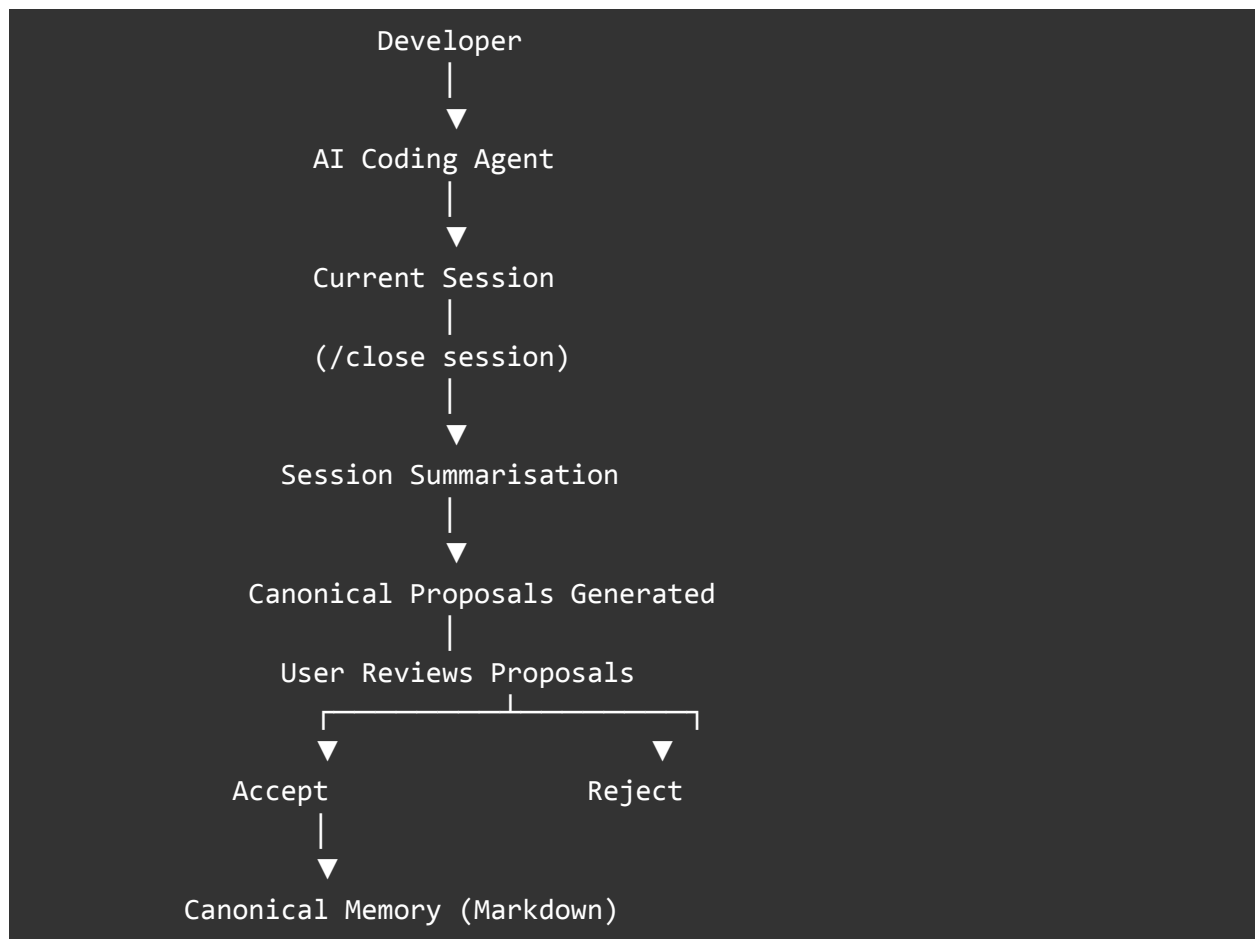
8) Evolution of EMS

The primary objective of this experiment was not to evaluate the current implementation of EMS, but to understand how an engineering memory system should be designed in the first place. Although EMS already demonstrated promising behaviour, the prompt-level analyses revealed several architectural assumptions that proved to be incomplete. Rather than simply refining the existing implementation, the experiment fundamentally changed our understanding of where engineering cognition should be extracted, how it should evolve over time, and when it should influence future reasoning.

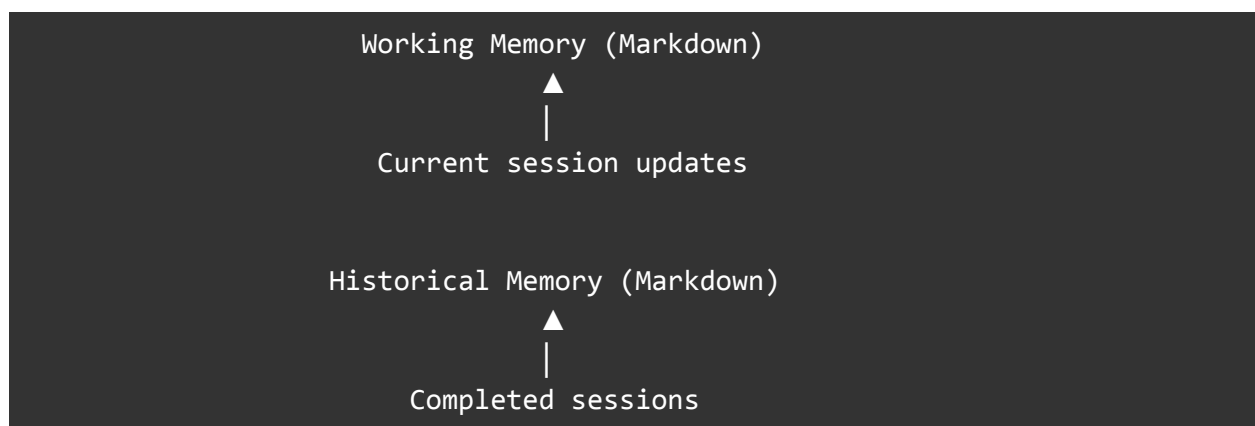
8.1) Initial EMS Design

Our original design treated engineering memory primarily as a **persistent knowledge repository**. The core assumption was that valuable engineering context could be extracted from completed coding sessions, stored as structured documentation, and later retrieved to initialise future coding sessions.

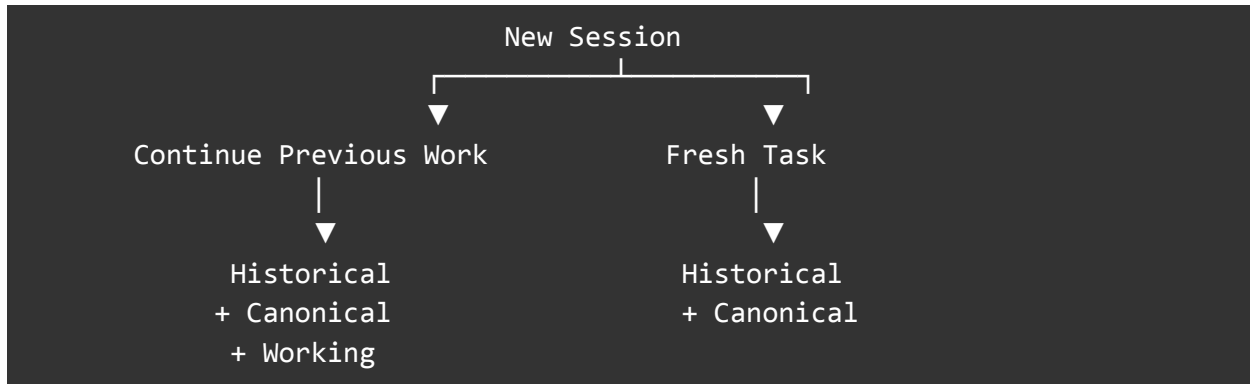
The simplified architecture was therefore:



with memory updates happening like this:



And memory injection happening like this:



This design already introduced several improvements over existing AI coding workflows by making engineering knowledge persistent across sessions and branches. However, the experiment exposed three major limitations.

1. **Knowledge was document-centric.** Engineering understanding was stored as documents rather than as evolving engineering concepts.
2. **Retrieval was eager.** Context was injected before reasoning began, increasing the risk of anchoring future investigations on outdated assumptions.
3. **Extraction was treated as a solved problem.** The system implicitly assumed that summarising an LLM conversation was sufficient to preserve engineering understanding.

The experimental results repeatedly challenged all three assumptions.

8.2) Lessons from the Experiment

One of the most significant outcomes of this experiment was that it fundamentally shifted our understanding of where the primary engineering challenge actually lies. The original EMS design implicitly assumed that once engineering knowledge had been extracted from a completed coding session, the remaining problems were largely those of storage and retrieval. Throughout the investigation, however, this assumption proved to be incomplete.

Across investigation, implementation, debugging and planning tasks, the quality of future reasoning depended far less on whether previous information had been preserved than on whether meaningful engineering understanding had first been extracted from the coding process itself. Equally important was *when* this accumulated understanding became available to the reasoning process. Repeatedly, repository verification produced better outcomes than eager context injection, suggesting that engineering cognition should support reasoning only after sufficient local understanding has been established.

The experiment also revealed that engineering understanding is inherently evolutionary. New observations rarely replaced previous knowledge outright; instead they refined, strengthened or superseded earlier engineering conclusions. Representing this process as independent markdown documents therefore became increasingly unnatural, as the system required

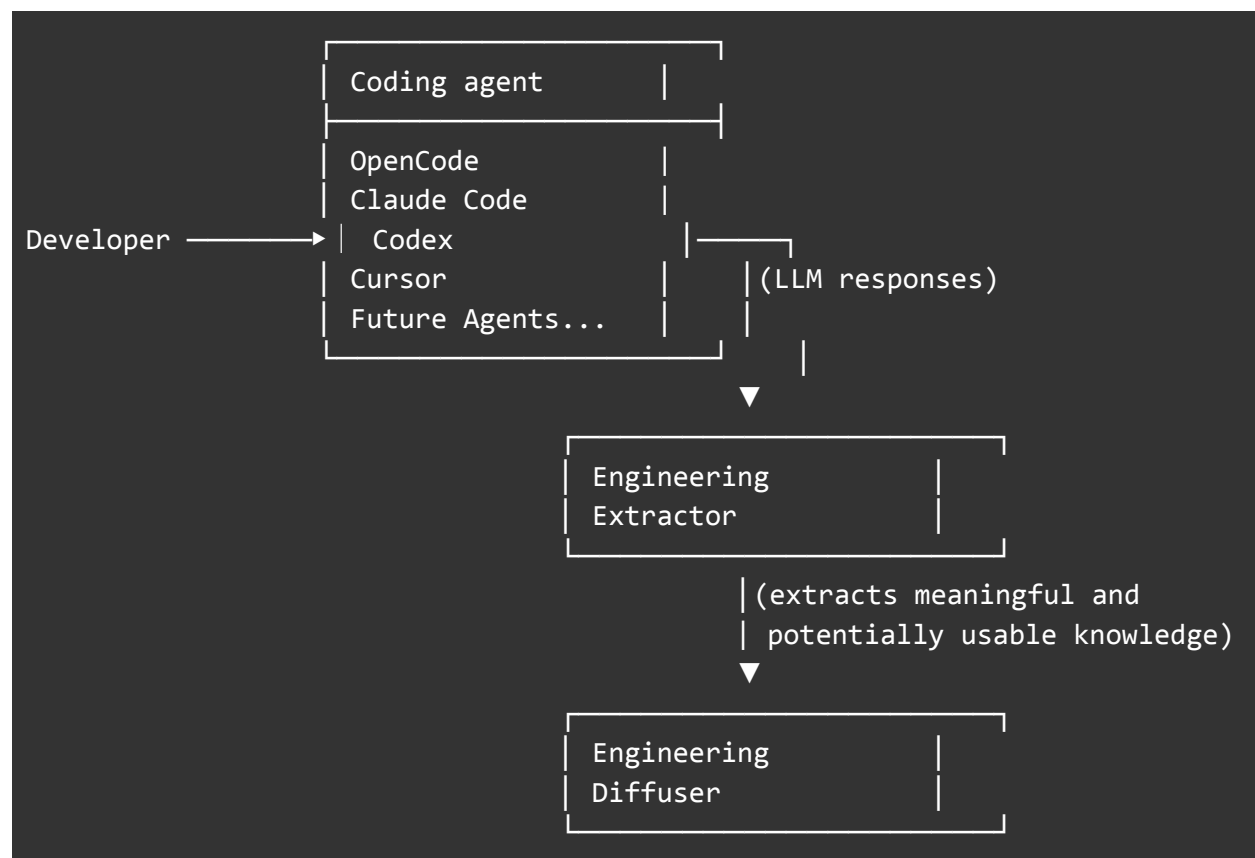
mechanisms for maintaining evolving engineering beliefs rather than merely accumulating static engineering notes.

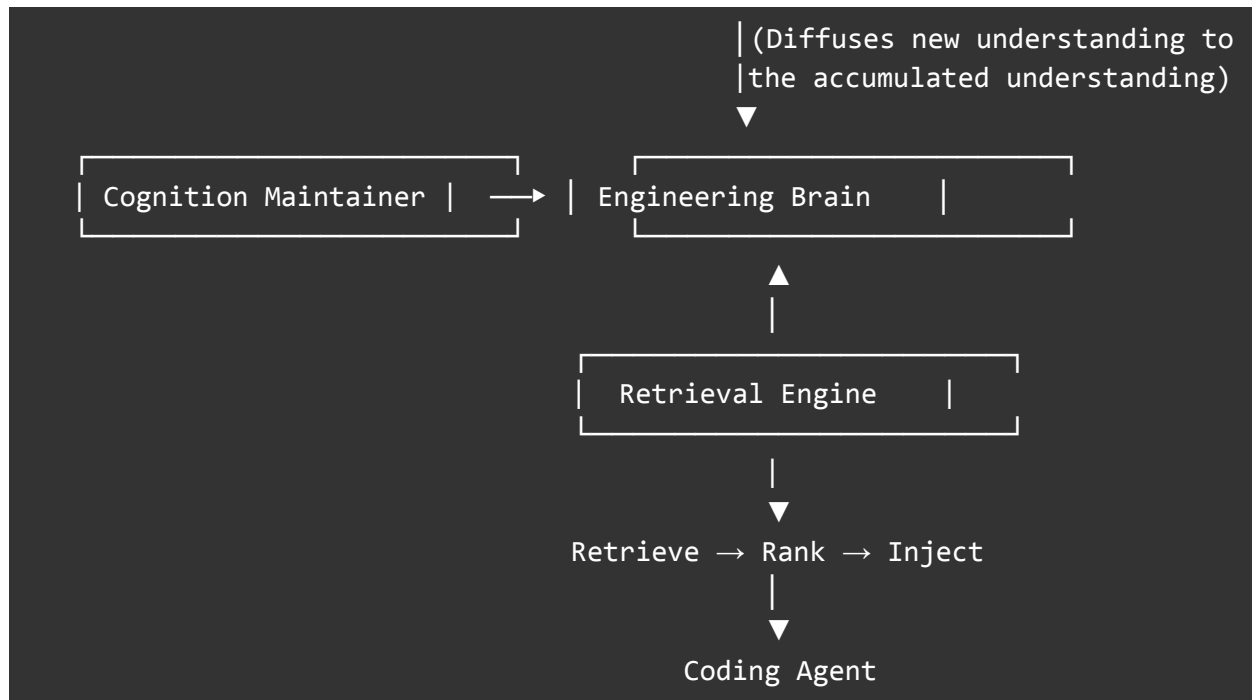
Taken together, these findings suggest that EMS should no longer be viewed as a persistent engineering memory system. Instead, it should be redesigned as a continuously evolving engineering cognition pipeline whose primary objective is to construct, maintain and retrieve engineering understanding rather than simply preserve engineering information.

8.3) Revised EMS Architecture

The experimental findings collectively suggest that engineering cognition should no longer be modelled as a persistent document store. Instead, it should be treated as a continuously evolving pipeline in which engineering understanding is progressively extracted, refined, maintained and retrieved throughout the lifetime of a software project. This perspective naturally separates EMS into four distinct research components: **Engineering Extraction, Engineering Diffusion, Cognition Maintenance, and Demand-driven Retrieval**.

Rather than directly persisting conversations or summaries, engineering understanding flows through these stages before becoming part of the long-term engineering knowledge available to future coding sessions. Each stage addresses a fundamentally different research problem and can evolve independently as better algorithms become available.





Unlike the original EMS architecture, where knowledge preservation itself was considered the primary objective, the revised pipeline treats persistent memory as an intermediate representation. The central objective instead becomes the continuous construction and evolution of engineering understanding.

Engineering Extractor

The experiment demonstrated that preserving engineering conversations is insufficient unless the underlying engineering understanding can first be recovered. Consequently, the Engineering Extractor becomes the first and arguably most important stage of the revised architecture.

Rather than generating summaries, the extractor attempts to identify reusable engineering artefacts hidden within LLM reasoning. These artefacts may represent architectural decisions, implementation strategies, debugging discoveries, repository assumptions, invariants, dependency relationships, performance observations, engineering hypotheses, or any other form of engineering understanding that may remain valuable beyond the current coding session.

Unlike the remaining components of EMS, extraction remains largely an open research problem. Accurately separating reusable engineering cognition from transient conversational reasoning is unlikely to emerge from prompt engineering alone and may require specialised extraction models trained using reinforcement learning or other task-specific optimisation techniques.

Engineering Diffuser

Individual engineering artefacts rarely exist in isolation. Instead, they continuously interact with previously accumulated understanding, strengthening existing conclusions, refining earlier assumptions, or replacing obsolete engineering knowledge. The experiment repeatedly exhibited this evolutionary behaviour, motivating the introduction of the Engineering Diffuser.

Rather than storing every extracted artefact independently, the diffuser determines how new engineering evidence should influence the existing knowledge base. A newly extracted observation may strengthen an existing engineering belief, contradict an earlier conclusion, supersede previous understanding, or introduce an entirely new engineering concept.

For example:

Belief v1

JSONL serialization dominates streaming overhead.



New observation

Module-level closure allocation also contributes.



Belief v2

Streaming overhead consists primarily of serialization cost and closure allocation.

Engineering cognition therefore becomes evolutionary rather than append-only. Instead of accumulating an ever-growing collection of independent engineering notes, EMS continuously refines its representation of the software system as new engineering evidence becomes available.

Engineering Brain

The original EMS architecture represented engineering knowledge primarily through hierarchical markdown documents. Although this design proved practical for human inspection, the experiment demonstrated that documents are poorly suited for representing evolving

engineering understanding. Engineering conclusions frequently overlap, supersede one another, or participate in relationships that cannot naturally be expressed through independent files.

The revised architecture therefore replaces the markdown wiki with an Engineering Brain built from structured engineering artefacts connected through semantic relationships. Artefacts become the smallest persistent unit of engineering cognition, allowing engineering understanding to evolve independently of any particular document representation.

Representative artefact categories include:

- Architecture
- Decision
- Hypothesis
- Invariant
- Implementation
- Bug
- Debugging Discovery
- Performance Finding
- Protocol
- API
- Behaviour
- Repository
- Assumption
- Experiment
- Benchmark
- Result

Documents may still be generated for human consumption whenever necessary, but they are no longer considered the canonical representation of engineering knowledge. Instead, they become derived views over a continuously evolving graph of engineering cognition.

Cognition Maintainer

The experimental findings also suggest that engineering cognition cannot be treated as static once it has been extracted. As software systems evolve, previously accumulated understanding gradually becomes outdated, contradictory, duplicated or incomplete. Without an explicit maintenance mechanism, the Engineering Brain would eventually degrade into an inconsistent collection of partially obsolete engineering artefacts.

To address this problem, the revised architecture introduces a dedicated Cognition Maintainer operating asynchronously alongside the developer's normal workflow. Rather than participating directly in code generation, this component continuously maintains the internal consistency of the Engineering Brain by updating semantic relationships between artefacts, recalculating confidence scores, merging duplicate engineering concepts, promoting or superseding

engineering beliefs as new evidence becomes available, and removing knowledge that is no longer considered valid.

Importantly, none of these operations occur synchronously with the developer's coding session. Once engineering artefacts have been extracted, cognition maintenance proceeds independently in the background, allowing the developer to continue working without waiting for the knowledge base to stabilise. This separation between interactive coding and long-running cognition maintenance reflects another important lesson from the experiment: engineering understanding evolves continuously, whereas engineering productivity depends on keeping the interactive feedback loop as short as possible.

Demand-driven Retrieval

One of the clearest observations emerging from the experiment was that retrieval quality depends less on *how much* engineering knowledge is available than on *when* that knowledge enters the reasoning process. Early retrieval frequently biased subsequent investigations, whereas repository verification followed by targeted retrieval consistently produced more reliable engineering decisions.

The revised EMS architecture therefore abandons eager context injection entirely. Rather than loading accumulated engineering knowledge before reasoning begins, retrieval becomes a reasoning operation in its own right. The coding agent is first allowed to establish local understanding of the current repository and implementation before requesting additional engineering cognition only when uncertainty arises.

This transforms the role of retrieval from a passive database lookup into an active component of the reasoning process. Retrieved artefacts are first ranked according to relevance, confidence and available context budget before being incorporated into the prompt presented to the coding agent. Consequently, engineering cognition supports reasoning only when it becomes beneficial, rather than attempting to initialise reasoning before sufficient local understanding has been established.

Implications

Taken together, these architectural changes represent considerably more than an incremental refinement of the original EMS implementation. They fundamentally change the role that engineering memory plays within the overall system.

The original EMS architecture viewed persistent memory as the primary objective: engineering sessions generated documents, documents were stored, and subsequent coding sessions consumed those documents. The experiment demonstrated that this framing captures only a small part of the underlying problem. Persistent storage alone does not produce accumulated

engineering understanding, nor does retrieving larger quantities of historical information necessarily improve engineering decisions.

The revised architecture instead treats engineering memory as an intermediate representation within a broader engineering cognition pipeline. Extraction, diffusion, maintenance and retrieval become independent research problems whose collective objective is not simply to preserve engineering information, but to continuously construct, refine and evolve engineering understanding over time. In this formulation, memory is no longer the endpoint of the system; it becomes one stage in the ongoing process through which software engineering knowledge gradually accumulates into persistent engineering cognition.

9) Threats to Validity

No empirical study is free from limitations, and the findings presented in this work should be interpreted within the context of the experimental design.

Single Repository.

The experiment was conducted on a single software repository and therefore reflects one particular engineering domain. Although the repository contains realistic architectural reasoning, implementation, debugging and planning tasks, the observed behaviour may differ for repositories with substantially different structures, programming languages or development workflows.

Single Model Family.

Both conditions were evaluated using the same underlying reasoning model. Consequently, the experiment isolates the effect of Engineering Memory Systems while minimising model variability, but it does not establish that the same observations necessarily generalise across different foundation models. Future work should evaluate the same benchmark using multiple coding agents and model families.

LLM-as-a-Judge Evaluation.

Prompt performance was assessed using an independent language model acting as the evaluation judge. Although the judge prompt remained fixed throughout the experiment and produced detailed, criterion-based assessments, language-model evaluation remains inherently probabilistic. The accompanying qualitative analysis was therefore used to validate the judge's numerical scores and to identify recurring behavioural patterns that might not be captured by quantitative evaluation alone.

Sequential Prompt Ordering.

The benchmark intentionally evaluates long-horizon engineering workflows in which each prompt builds upon the previous one. While this design is essential for measuring engineering cognition, it also introduces ordering effects. Errors introduced early in the workflow may influence later prompts, making individual prompt scores partially dependent on previous reasoning quality.

Sample Size.

The benchmark consists of thirty prompts spanning five engineering activities. Although this was sufficient to expose several consistent behavioural patterns, the sample remains relatively small. A larger benchmark covering multiple repositories and substantially longer engineering sessions would provide stronger statistical confidence.

Evolving EMS Design.

The Engineering Memory System evaluated in this work represents an early research prototype rather than a final system architecture. In fact, one of the principal outcomes of this experiment was the redesign of EMS itself—from a document-oriented memory system towards an Engineering Cognition pipeline. Consequently, the reported results should be viewed as evaluating the current generation of EMS rather than establishing an upper bound on what engineering cognition systems can ultimately achieve.

10) Future Work

Open Research Questions

The objective of this work was not to produce a complete Engineering Cognition system, but rather to investigate whether accumulated engineering understanding can measurably influence software engineering performance. Although the experiment demonstrated that persistent engineering cognition can improve architectural reasoning, implementation continuity and long-horizon planning, it also exposed a number of fundamental questions that remain unanswered.

Unlike conventional software engineering problems, these questions cannot be addressed solely through implementation. They concern the representation, evolution and utilisation of engineering understanding itself, and therefore define the immediate research agenda for Engineering Cognition.

Engineering Understanding Extraction.

How can reusable engineering understanding be extracted reliably from the intermediate reasoning produced by large language models rather than from their final textual responses? More importantly, how can such understanding be distinguished from temporary reasoning that should not become part of persistent engineering cognition?

Engineering Belief Formation.

How should individual engineering artefacts be combined into higher-level engineering beliefs without introducing incorrect generalisations or discarding repository-specific context? What principles govern when multiple observations should become a single piece of accumulated engineering understanding?

Belief Evolution.

Engineering understanding changes continuously as software evolves. How should engineering

beliefs evolve when new evidence becomes available? Should beliefs be versioned, superseded, merged, or continuously refined, and how should contradictory observations be represented without losing historical reasoning?

Engineering Ontologies.

What ontology best represents engineering cognition? Which primitive artefact categories—such as architectural decisions, implementation strategies, debugging discoveries, invariants, protocols, performance observations and repository assumptions—should exist, and how should semantic relationships between them emerge over time?

Cognition Maintenance.

How should an engineering cognition system continuously maintain accumulated understanding? Relationship discovery, duplicate detection, confidence estimation, ontology evolution and belief consolidation may all require independent maintenance processes, but the appropriate maintenance strategy remains unknown.

Retrieval Timing.

This experiment suggests that retrieval timing may be as important as retrieval quality. Rather than retrieving all available knowledge before reasoning begins, engineering cognition may need to be injected only when uncertainty naturally arises during problem solving. Determining the optimal retrieval policy remains an open research problem.

Model Independence.

Should engineering cognition be extracted using whichever coding model the developer is currently using, or should a specialised extraction model—potentially trained using reinforcement learning—be responsible for constructing persistent engineering understanding independently of the coding agent itself?

Knowledge Representation.

Should engineering cognition ultimately be represented as markdown documents, structured engineering artefacts, semantic knowledge graphs, engineering beliefs, or an entirely different representation? The experiment indicates that documents are useful for human inspection, but they may not constitute the most appropriate internal representation for accumulated engineering understanding.

Cross-Repository Cognition.

Can engineering understanding learned within one repository transfer meaningfully to another repository operating in a similar engineering domain? If so, which abstractions become repository-independent and which remain inherently local?

Measuring Engineering Cognition.

Finally, how should Engineering Cognition itself be evaluated? Current benchmarks, including EC-Bench, infer accumulated understanding indirectly through downstream engineering performance. Future benchmark design may instead require evaluation methodologies capable of measuring engineering understanding as a first-class computational capability.

Collectively, these questions define the immediate research agenda for Engineering Cognition. Rather than representing incremental improvements to EMS, they expose fundamental challenges concerning how engineering understanding should be extracted, represented, maintained and reused over time. The answers to these questions will ultimately determine the design of future Engineering Memory Systems and guide the continued evolution of EC-Bench as a benchmark for evaluating persistent engineering cognition.

11) What We Believe Today

The purpose of this work was never to demonstrate that one engineering memory system outperformed another. Instead, it was to understand what persistent engineering understanding actually requires.

Before conducting this experiment, we viewed the central challenge as one of memory persistence. We assumed that if engineering knowledge could simply be retained across coding sessions, future software engineering agents would naturally become more capable over time. EMS was therefore originally conceived as a structured memory layer that organised engineering documentation through hierarchical markdown files and made that information available to future sessions.

The experimental findings substantially changed this perspective.

Across investigation, architectural reasoning, implementation, debugging and planning tasks, improvements were rarely produced by storing more information. Instead, improvements emerged when previously accumulated understanding was reused appropriately, updated when necessary, and retrieved only when reasoning demanded it. Conversely, several regressions occurred not because information was unavailable, but because previously accumulated understanding was applied prematurely or expanded beyond the scope of the current engineering task.

These observations suggest that the limiting factor is not persistent memory, but the continuous construction and evolution of engineering understanding.

Accordingly, our view of EMS has changed in several important ways.

Before the experiment	What we believe today
The problem is persistent memory.	The problem is persistent engineering cognition.
Better retrieval produces better engineering.	Better extraction and belief formation produce better engineering.
Markdown documents are the primary	Structured engineering artefacts and evolving

knowledge representation.	engineering beliefs should become the canonical representation.
Knowledge should be retrieved at the beginning of a session.	Knowledge should be retrieved only when reasoning requires it.
Memory accumulation is the objective.	Understanding accumulation is the objective.
EMS is primarily a memory management system.	EMS should become a continuously evolving engineering cognition pipeline.
Repository context is sufficient.	Reasoning discipline determines whether accumulated understanding improves future work.

Perhaps the most significant outcome of this work is that it reframes the research problem itself.

We no longer view EMS as the destination of this research. Instead, EMS has become a vehicle for investigating a broader question:

Can engineering understanding itself become a computational object that accumulates, evolves and improves software engineering over time, independently of any individual coding agent?

This experiment does not answer that question.

It merely provides evidence that the question is worth pursuing.

Appendix

Appendix A - Complete Prompt Set

All the prompts were executed using the [Kimi K2.5](#) model provided with OpenCode Zen. The complete EC-Bench prompt set contains 30 engineering prompts spanning investigation, architectural reasoning, implementation, debugging and planning tasks. Due to length, the complete prompt corpus is available in the accompanying repository. Here's an example:

Prompt 1:

Phase: Investigation

Objective: Orient yourself in the repository and identify the core modules that participate in the HTTP request path.

Prompt:

I just joined the FastAPI team. Before I touch any code, I need to understand the repository layout. Walk me through the responsibilities of `fastapi/routing.py`, `fastapi/dependencies/utils.py`, `fastapi/applications.py`, and `fastapi/openapi/utils.py`. For each module, tell me the single most important class or function that an incoming HTTP request flows through.

Expected engineering knowledge accumulated after completing the prompt:

Understanding of the four pillars of FastAPI (routing, DI, app factory, OpenAPI) and which symbols are on the critical path.

Why this prompt naturally follows from previous work:

This is the foundational orientation step; nothing precedes it.

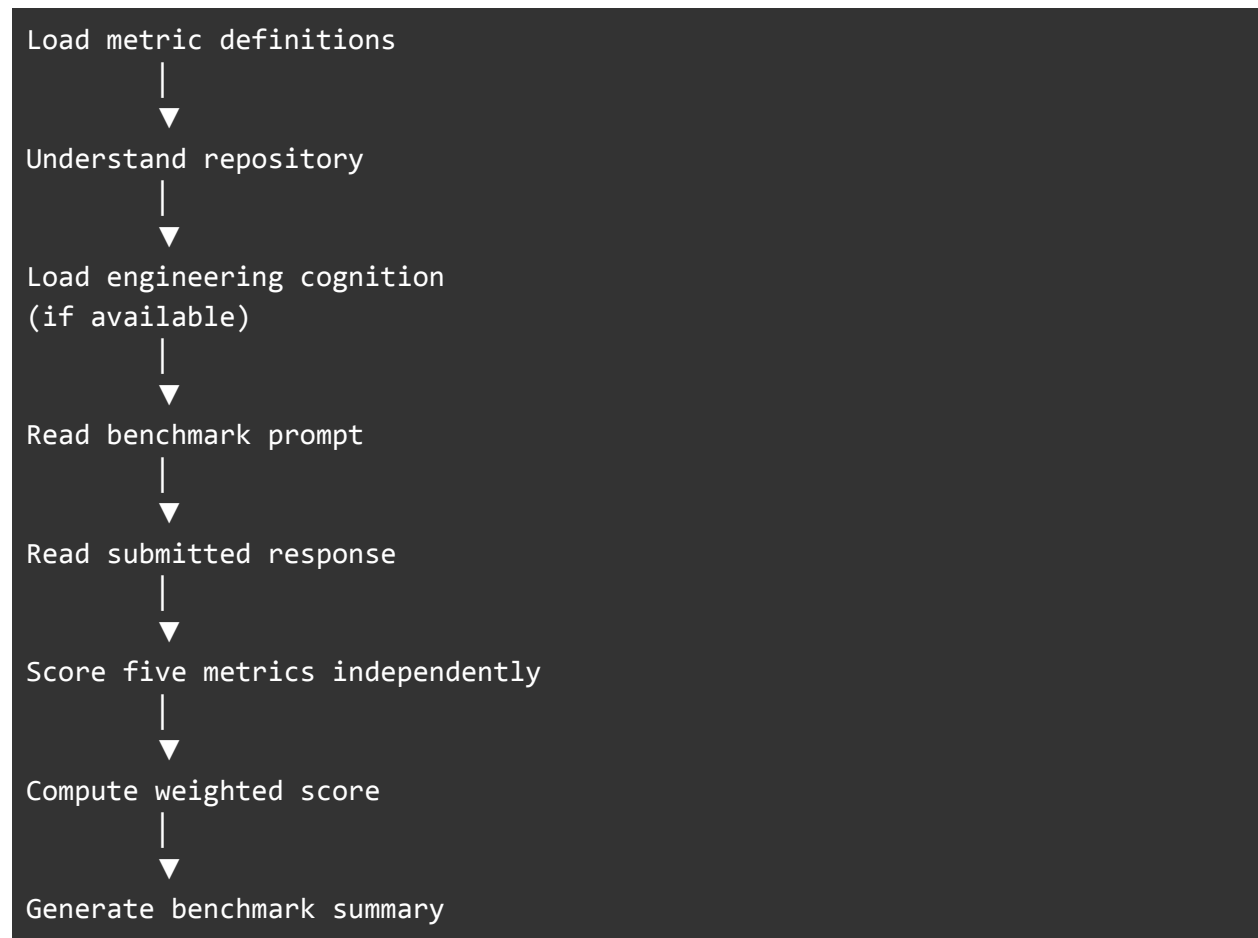
The remaining prompts can be found [here](#).

Appendix B - Judge Prompt

The judge prompt for the LLM responses were executed using the [GLM-5.2](#) model provided with OpenCode Zen. Each benchmark response is evaluated under a repository-first evaluation policy. The judge has access to:

- repository state
- benchmark prompt
- submitted engineering response
- engineering cognition (when enabled)
- metric definitions

The judge evaluates every response independently and produces continuous scores across the five benchmark metrics. The complete Judge implementation, evaluation prompt and scoring logic are available [here](#). This was the evaluation pipeline:



The metric weights used during the experiment were as follows:

Metric	Weight
Architectural Continuity	0.30
Engineering Cognition Reuse	0.30
Repository Groundedness	0.15
Engineering Quality	0.15
Debugging Investigation Efficiency	0.10

Hence, overall score was calculated as follows:

$(\text{Architectural Continuity} * 0.30) + (\text{Engineering Cognition Reuse} * 0.30) + (\text{Repository Groundedness} * 0.15) + (\text{Engineering Quality} * 0.15) + (\text{Debugging Investigation Efficiency} * 0.10)$

Appendix C - Metric Definitions

Metrics used to score the LLM responses have already been talked about earlier. The complete prompt can be found [here](#).

Appendix D - Full Prompt-Level Results

Here's the table with detailed prompt level scores for each metric:

Prompt Number	Metric	With EMS	Without EMS	Delta
1	Architectural Continuity	8.5	8.5	0
	Repository Groundedness	9	9	0
	Engineering Cognition Reuse	7.5	5	2.5
	Engineering Quality	8.5	8.5	0
	Debugging Investigation Efficiency	8	7.5	0.5
2	Architectural Continuity	9	9.5	-0.5
	Repository Groundedness	9.5	9.5	0
	Engineering Cognition Reuse	8.5	5	3.5
	Engineering Quality	8.5	9	-0.5
	Debugging Investigation Efficiency	8.5	8.5	0
3	Architectural Continuity	9	9.5	-0.5
	Repository Groundedness	9.5	9.5	0
	Engineering Cognition Reuse	9	5	4
	Engineering Quality	8.5	9	-0.5
	Debugging Investigation Efficiency	8.5	8	0.5
4	Architectural Continuity	9	9	0
	Repository Groundedness	9	9	0

	Engineering Cognition Reuse	8.5	5	3.5
	Engineering Quality	8	8.5	-0.5
	Debugging Investigation Efficiency	8.5	8	0.5
5	Architectural Continuity	8.6	9.5	-0.9
	Repository Groundedness	9.3	9.5	-0.2
	Engineering Cognition Reuse	8.8	5	3.8
	Engineering Quality	8.7	9	-0.3
	Debugging Investigation Efficiency	8.5	8.5	0
6	Architectural Continuity	7.5	9	-1.5
	Repository Groundedness	7	9.5	-2.5
	Engineering Cognition Reuse	7	5	2
	Engineering Quality	6.5	8.5	-2
	Debugging Investigation Efficiency	7	8.5	-1.5
7	Architectural Continuity	9	9	0
	Repository Groundedness	8.2	8.5	-0.3
	Engineering Cognition Reuse	9.1	5	4.1
	Engineering Quality	8.5	9	-0.5
	Debugging Investigation Efficiency	8	8	0
8	Architectural Continuity	9.2	9	0.2
	Repository Groundedness	9.5	9	0.5
	Engineering Cognition Reuse	9.3	5	4.3
	Engineering Quality	8.8	8.5	0.3
	Debugging Investigation Efficiency	9	8.5	0.5
9	Architectural Continuity	9.1	9.5	-0.4
	Repository Groundedness	7.8	9.5	-1.7
	Engineering Cognition Reuse	9.3	5	4.3
	Engineering Quality	7.5	9	-1.5
	Debugging Investigation Efficiency	8	8.5	-0.5
10	Architectural Continuity	9.4	9	0.4
	Repository Groundedness	8.6	9.5	-0.9
	Engineering Cognition Reuse	9.5	5	4.5
	Engineering Quality	8.3	9	-0.7

	Debugging Investigation Efficiency	8.5	8.5	0
11	Architectural Continuity	9	9.5	-0.5
	Repository Groundedness	8.8	9.5	-0.7
	Engineering Cognition Reuse	9.2	5	4.2
	Engineering Quality	8	9	-1
	Debugging Investigation Efficiency	8.5	8.5	0
12	Architectural Continuity	9.5	9	0.5
	Repository Groundedness	9.4	9.5	-0.1
	Engineering Cognition Reuse	9.6	5	4.6
	Engineering Quality	9.1	8.5	0.6
	Debugging Investigation Efficiency	9.2	8.5	0.7
13	Architectural Continuity	9.3	9	0.3
	Repository Groundedness	9	8.5	0.5
	Engineering Cognition Reuse	9.4	5	4.4
	Engineering Quality	8.8	8.5	0.3
	Debugging Investigation Efficiency	8.5	8	0.5
14	Architectural Continuity	9.2	9	0.2
	Repository Groundedness	9.3	7.5	1.8
	Engineering Cognition Reuse	9.5	5	4.5
	Engineering Quality	8.7	7.5	1.2
	Debugging Investigation Efficiency	8.8	8	0.8
15	Architectural Continuity	8.7	9.5	-0.8
	Repository Groundedness	9.5	9	0.5
	Engineering Cognition Reuse	9	5	4
	Engineering Quality	8.5	8.5	0
	Debugging Investigation Efficiency	8	8	0
16	Architectural Continuity	8.5	9.2	-0.7
	Repository Groundedness	9	9	0
	Engineering Cognition Reuse	9.3	8.7	0.6
	Engineering Quality	8.5	8.5	0
	Debugging Investigation Efficiency	8	8	0
17	Architectural Continuity	9	9.3	-0.3

	Repository Groundedness	9.3	9.1	0.2
	Engineering Cognition Reuse	9.5	9	0.5
	Engineering Quality	9	8.8	0.2
	Debugging Investigation Efficiency	8.5	8.5	0
18	Architectural Continuity	9	9.5	-0.5
	Repository Groundedness	8.8	9.2	-0.4
	Engineering Cognition Reuse	9.2	9.2	0
	Engineering Quality	8.5	8.7	-0.2
	Debugging Investigation Efficiency	8	8	0
19	Architectural Continuity	9	8.5	0.5
	Repository Groundedness	8.5	8.3	0.2
	Engineering Cognition Reuse	8.8	7.8	1
	Engineering Quality	7.5	7	0.5
	Debugging Investigation Efficiency	7.5	7.5	0
20	Architectural Continuity	8.5	9	-0.5
	Repository Groundedness	9	9.2	-0.2
	Engineering Cognition Reuse	9	8.8	0.2
	Engineering Quality	8	8.5	-0.5
	Debugging Investigation Efficiency	8.5	8.5	0
21	Architectural Continuity	8	7	1
	Repository Groundedness	9.5	9	0.5
	Engineering Cognition Reuse	8	6	2
	Engineering Quality	8	8.5	-0.5
	Debugging Investigation Efficiency	8.5	8.5	0
22	Architectural Continuity	7	7	0
	Repository Groundedness	7	9	-2
	Engineering Cognition Reuse	8	6.5	1.5
	Engineering Quality	7	8	-1
	Debugging Investigation Efficiency	8.5	7.5	1
23	Architectural Continuity	5	8	-3
	Repository Groundedness	3.5	9.5	-6
	Engineering Cognition Reuse	5.5	8.5	-3

	Engineering Quality	4	9	-5
	Debugging Investigation Efficiency	4	9.5	-5.5
24	Architectural Continuity	6	8.5	-2.5
	Repository Groundedness	7	9	-2
	Engineering Cognition Reuse	7.5	8.8	-1.3
	Engineering Quality	4	8.5	-4.5
	Debugging Investigation Efficiency	4	7	-3
25	Architectural Continuity	8	8.5	-0.5
	Repository Groundedness	8	7.5	0.5
	Engineering Cognition Reuse	7	8	-1
	Engineering Quality	7.5	9	-1.5
	Debugging Investigation Efficiency	8.5	8	0.5
26	Architectural Continuity	9	8.5	0.5
	Repository Groundedness	6.5	7.5	-1
	Engineering Cognition Reuse	8.5	7.5	1
	Engineering Quality	7.5	8.5	-1
	Debugging Investigation Efficiency	8.5	8	0.5
27	Architectural Continuity	8	7	1
	Repository Groundedness	5.5	8	-2.5
	Engineering Cognition Reuse	6	7.5	-1.5
	Engineering Quality	5.5	7	-1.5
	Debugging Investigation Efficiency	5	6.5	-1.5
28	Architectural Continuity	9	8.5	0.5
	Repository Groundedness	8.7	8.5	0.2
	Engineering Cognition Reuse	9.2	6	3.2
	Engineering Quality	8.5	8	0.5
	Debugging Investigation Efficiency	8.8	7.5	1.3
29	Architectural Continuity	9.3	8.5	0.8
	Repository Groundedness	9	8.5	0.5
	Engineering Cognition Reuse	9.5	7	2.5
	Engineering Quality	8.5	7.5	1
	Debugging Investigation Efficiency	9	7.5	1.5

30	Architectural Continuity	9	8.5	0.5
	Repository Groundedness	9.2	8.5	0.7
	Engineering Cognition Reuse	9.5	7	2.5
	Engineering Quality	8.5	7.5	1
	Debugging Investigation Efficiency	9	7.5	1.5